
Docente del curso: Benner Humberto Ojeda Rios _____

Nombres y Apellidos del estudiante: _____

Fecha: 29/01/2025 _____ Sección: 1 _____ Nota: _____

Indicaciones específicas:

- Duración: **30** minutos
 - En caso de haber soluciones similares de preguntas serán calificados con cero (0).
-

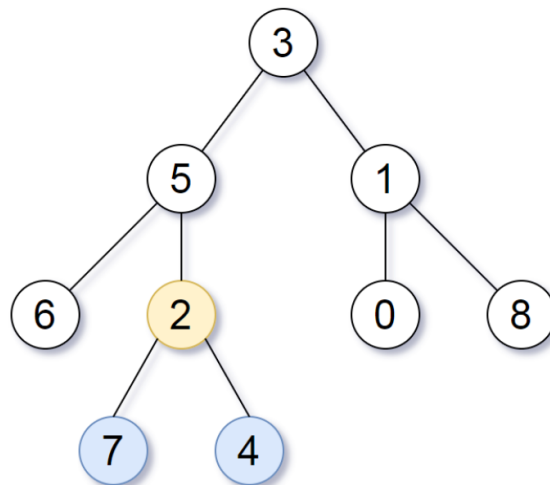
Resuelva:

1. (6 pts) Dada la raíz de un árbol binario, devuelve el ancestro común más bajo de sus hojas más profundas.

Recordar que:

- El nodo de un árbol binario es una hoja si y sólo si no tiene hijos.
- La profundidad de la raíz del árbol es 0. Si la profundidad de un nodo es d , la profundidad de cada uno de sus hijos es $d + 1$.
- El ancestro común más bajo de un conjunto S de nodos es el nodo A con la mayor profundidad, de modo que cada nodo en S está en el subárbol con raíz A .

Ejemplo 1:



- **Entrada:** root = [3,5,1,6,2,0,8,null,null,7,4]
- **Salida:** [2,7,4]
- **Explicación:** Devolvemos el nodo con valor 2, coloreado en amarillo en el diagrama.

Los nodos coloreados en azul son los nodos de las hojas más profundos del árbol. Tenga en cuenta que los nodos 6, 0 y 8 también son nodos hoja, pero su profundidad es 2, pero la profundidad de los nodos 7 y 4 es 3.

Ejemplo 2:

- **Entrada:** root = [1]
- **Salida:** [1]
- **Explicación:** La raíz es el nodo más profundo en el árbol, y es el ancestro común más bajo (LCA) de sí mismo.

Ejemplo 3:

- **Entrada:** root = [0,1,3,null,2]
- **Salida:** [2]
- **Explicación:** La hoja más profunda en el árbol es 2. El ancestro común más bajo (LCA) de un único nodo es el mismo nodo.

Restricciones:

- El número de nodos en el árbol estará en el rango [1, 1000].
- $0 \leq \text{Nodo.val} \leq 1000$
- Los valores de los nodos del árbol son únicos.

```
/** TEMPLATE
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x),
 *                                                         left(left),
 *                                                         right(right) {}
 * };
 */
```

```
class Solution {  
public:  
    TreeNode* solve(TreeNode* root) {  
  
    }  
};
```